Campus Lost & Found

Team 10 - INF2006

Team Members: Chern De Qian (2502760), Teo Siow Min Glenda (2501813), Zhou Xinlong (2500775), Liew Jia Rong (2501090), Ruan Zhongjiang (2501780), Thiha Thein (2502266)

1. Problem, users and success criteria

1.1 Users and problem

Our users are SIT students and staff who lose or find belongings on campus, together with the security team, who receive and hold found items and must return them to the right owner. In the application, students and staff would have to sign up with a student role, while security staff use the admin role.

The current process in SIT has no designated lost and found area on campus. Finders are told to hand items to Security, either by approaching a Security Officer on patrol or by visiting the Security Room at W3. Owners must then visit Security or email the SIT Estate Security team to ask about their item. This leaves the process fragmented. Notices appear across chat groups and physical noticeboards, owners do not know where to look, and finders do not know whom to contact. Staff have no searchable record, so items go unclaimed and staff spend time on manual enquiries.

Our service would give everyone one place to report a lost item, report a found item, browse what has been handed in, and submit an ownership claim. Its key feature is similarity-based matching. As soon as the owner submits a lost item, the system would compare the category, description and location against every open found report and show a ranked list of likely matches with a match percentage (e.g. “Match score: 72%”). Owners no longer have to scroll through every notice. At most five matches are shown, and only those scoring 20% or more, so weak one-word overlaps are filtered out. Matching never releases an item: the finder records a private detail that only Security sees, and Security compares it with the claimant's answer before approving a claim.

1.2 Success criteria

These are the targets set for the campus lost & found project

Success Criterion	Measures / Targets	Evidence
-Core workflows work without errors on the deployed site -Covers register, log in, report lost, report found, browse, view matches, submit claim, admin approve/reject, log out	-100% of test steps pass -Tested through the ALB URL	-Test 1 – functional test -evidence/test-functional.md
-Correct found item shows near the top of the list -Each match shows a match percentage	-Correct item in the top 5 for at least half of lost items (Top 5 accuracy ≥ 50%) -Every match shows a percentage Only matches of 20% or more are shown	-Test 3 – data/AI test -evidence/test-data-ai.md -evidence/test-data-ai-output.csv
-No user can see or change another user's reports, matches or claims	-0 successful attempts when changing item IDs in the URL -0 successful attempts when changing item IDs in form fields	-Test 2 – security test -evidence/test-security.md

-Problems or abuse can be spotted by the team	-CloudWatch alarm goes off within 5 minutes of a fault we cause on purpose -Example: stopping an EC2 instance	-Test 4 – scale /resilience test -evidence/test-resilience.md

2. Solution overview and architecture

2.1 Solution Summary

Campus Lost & Found is built as a three-tier PHP web application hosted on AWS. Traffic coming into the site first hits an Application Load Balancer (lostfound-alb), which sits in two public subnets and spreads it across a pool of EC2 web servers (lostfound-web). These run in private app subnets (no public IP) in two Availability Zones and are managed by an Auto Scaling group (lostfound-asg) so the pool can grow or shrink with demand. Every instance is built by the launch template (lostfound-lt) running src/infra/user-data.sh, and reaches the internet only outbound through a NAT gateway. Each instance runs Apache with PHP (php-fpm) serving the pages in src/public/, with shared helpers in src/inc for database access (db.php), session-based authentication (auth.php) and matching (matching.php).

All persistent data (users, lost/found reports and claims) goes into a private RDS MySQL database (lostfound-db), using the schema in data/schema.sql. The matching feature is a TF-IDF + cosine similarity engine written in Python that compares category + description + location (analytics/match_items.py) and runs on the same EC2 instances. When a user opens their matches, matching.php pulls every open found item from RDS and passes the data to the script via shell_exec(). The script returns the top 5 found items ranked by similarity score, PHP drops anything below 0.20, and matches.php displays each remaining item with its score as a percentage. The same function powers the “Possible matches” counter on the dashboard. CloudWatch collects ALB, EC2 and RDS metrics, the alarm lostfound-unhealthy-alarm and Apache/PHP logs in the log group /lostfound/httpd, while an AWS Budgets alert watches the cost.

2.2 Trust boundaries and data flow

The architecture diagram in Appendix Figure 1 marks three trust boundaries. (1) Internet → AWS: only the ALB accepts inbound traffic (alb-sg: HTTP 80 from anywhere); the NAT gateway is outbound only. (2) ALB → web tier: web-sg accepts HTTP 80 only from alb-sg. (3) Web tier → database: db-sg accepts MySQL 3306 only from web-sg, and lostfound-db is not publicly accessible. A request flows user → ALB → lostfound-web (PHP) → RDS; for matches, PHP also calls the local Python matcher and returns ranked cards to the user.

3. Cloud service/deployment choices and trade-offs

3.1 Chosen service and deployment model

Service model. We chose a mixed model. Compute and networking are IaaS (EC2, VPC, ALB), so under the shared responsibility model we manage the OS, runtime, application and data, while AWS manages the hypervisor, hardware and facilities. The database and monitoring are PaaS (RDS, CloudWatch), so AWS also patches the database engine and OS and runs backups, and we only manage the schema, accounts and settings.

Deployment model. Public cloud (AWS, region us-east-1 through the AWS Academy Learner Lab), single region across two Availability Zones. A private cloud would give SIT more control over data but needs hardware, staff and has limited scalability; hybrid or

multi-cloud would add integration and management complexity with no benefit for one small campus service. Public cloud gives pay-per-use pricing and elastic scaling, and the system stores a limited amount of personal data compared with higher-risk systems such as financial or medical services.

Layer	Services	Model	Our responsibility
Compute	EC2 (Apache + PHP, Python 3 + scikit-learn for matching) in an Auto Scaling group	IaaS	OS, runtime, app deployment, instance type, scaling limits
Network and load balancing	VPC, public, private app and private DB subnets across 2 AZs, NAT gateway, security groups, ALB	IaaS	Segmentation, inbound/outbound rules, health checks
Database	RDS MySQL	PaaS (managed DB)	Schema, queries, access, automated backups (1-day retention)
Secrets and access	EC2 IAM role (LabInstanceProfile, used only by the CloudWatch agent); DB credentials in src/.env written at boot by the launch template (no Secrets Manager)	IaaS (config file) / PaaS (IAM)	Restrict the IAM role to required permissions; protect the config file and keep credentials out of the repository
Monitoring	CloudWatch logs, metrics and alarms; AWS Budgets	PaaS	Choosing metrics, alarm thresholds, log retention (7 days), budget limit
Analytics	TF-IDF matching, in-app on EC2	Runs on our IaaS	Matching code, dataset, evaluation

3.2 Alternatives considered and rejected

- **Elastic Beanstalk / managed app platform (PaaS).** Beanstalk could deploy the PHP app with load balancing and Auto Scaling quickly, but it would hide much of the scaling and health-check configuration that we needed to implement and test ourselves. EC2 gave us more control and also allowed Python and scikit-learn to run beside PHP.

- **Serverless (Lambda + API Gateway + DynamoDB).** This could be cheap at campus scale, but our claims workflow relies on relational data and multi-record updates. Moving the existing PHP application to Lambda would also require a major rewrite.

- **Image-based matching.** This would require users to have photos of both lost and found items, plus S3, an AI service and labelled image data. Text descriptions are always available, and TF-IDF runs on our existing EC2 instances without additional AI-service cost.

3.3 Assumptions, constraints, workload and cost control

- **Expected workload:** The system is for one campus, so we expect somewhere between tens and a few hundred open reports at any time. Traffic will probably go up at the start of each term and after big events. Most of the requests are just people loading pages, so small burstable instances should be enough.

- **Matching uses the same CPU as the website:** PHP runs the Python TF-IDF matcher whenever matches or dashboard results are requested. This is acceptable at campus scale but becomes slower as data and traffic increase. For a larger deployment, we would calculate matches in a background job and store the scores.

- **Instance dependencies:** Each EC2 instance needs Python 3 and scikit-learn. The launch template installs them through user-data.sh, using the NAT gateway because the instances have no public IPs.

- **Sessions:** PHP sessions are stored locally on each instance, so we enabled one-day target-group stickiness. A production system should instead use shared session storage.

- **Cost control:** We used t3.micro instances, and the Auto Scaling group is set to a minimum of 1, desired 2 and maximum 3. The database is a single-AZ db.t3.micro RDS instance. Multi-AZ would roughly double the database cost, which we didn't think was worth it for a school project. There are no S3 or Secrets Manager costs, since we no longer store images and the database credentials are kept in a config file on the instance. A single NAT gateway (one AZ) serves both private app subnets. We also set CloudWatch logs to be kept for 7 days. We created a US$20 monthly AWS Budget with a forecast alert. By 11 October, AWS forecast monthly spending of about US$32.90, above the budget limit, so the forecast alert was triggered.

4. Implementation, data design and security controls

#	Workflow	Actor	Summary
W1	Register and log in	Student/staff	Create an account with name, email and password, then log in. New accounts always get the student role.
W2	Report a lost item	Student/staff	Submit category (fixed dropdown), description, location and date lost. After saving, the user is sent straight to their possible matches
W3	Register a found item	Student/staff/Security	Submit the same four fields plus a private distinguishing detail (e.g. a sticker underneath) that only Security will see. A confirmation page follows
W4	Search and browse	Any signed-in user	Filter by category, location and date or use keyword search on descriptions. Only open found items are listed.
W5	View description matches	Owner	Top 5 open found items ranked by similarity of category + description + location, each with a match

			percentage; items below 20% are hidden
W6	Submit an ownership claim	Owner	Click "This is mine, claim it" on a match (W5) and answer a verification question. A pending claim is recorded; no item status changes until Security approves it (W8). Claimant and finder track it in My Claims.
W7	Dashboard	Any logged in user	Shows the user's own reports and counts of lost items, found items, possible matches (≥ 20%) and pending claims
W8	Review claims	Security (Admin)	Admin sees the claimant's answer next to the finder's private detail and approves or rejects. Approval marks the found item "matched", the lost item "claimed" and auto-rejects other pending claims on that item.

Input validation is done on the server, because browser checks can be bypassed (tests S07, S16): every form field is required and trimmed; categories come from a fixed list; IDs are cast with intval() and all queries use prepared statements; all user text is escaped with htmlentities() on output; passwords need 8+ characters with upper, lower, number and symbol (register.php).

4.1 Data design

The relational schema (data/schema.sql, fields in data/DATA_DICTIONARY.md) has three tables. users holds name, unique email, bcrypt password_hash and a role (student/admin). items holds both lost and found reports (type ENUM) with category, description, location, date, reported_by (foreign key used in every ownership check), the found-only private_detail, and a status lifecycle (open → matched/claimed on approval). claims links a lost item, a found item and the claimant, with the verification answer and a pending/approved/rejected status. We chose a relational database (RDS MySQL) over NoSQL because claims join three records and approval updates several rows together; at campus scale vertical scaling of one small instance is enough. No images are stored, so no S3 bucket is needed; the instances use EBS only for their boot volumes. The unused photo_url column is reserved for future photo upload.

4.2 Security controls

Following the shared responsibility model, AWS secures the cloud (data centres, hardware, hypervisor, RDS engine patching) and we secure what we put in it. Every control below is mapped to a threat and a test in evidence/threat-control-map.md (T1–T18).

- **Authentication:** bcrypt password hashing (S17), the same error for a wrong password or unknown email (S12), and session_regenerate_id() at login against session fixation. New accounts are always student; admin is set only in the database.
- **Authorisation:** every query on user-owned data includes reported_by = <session user>; claim.php re-checks ownership server-side; admin.php checks the role before reading data or handling a POST (S01–S10). The finder's private detail is never selected by student pages (S11).

- **Least privilege:** the app connects as lostfound_app with only SELECT, INSERT and UPDATE on the three tables (data/db_app_user.sql; no DELETE, DROP or GRANT). The RDS master account is used only to load the schema. The EC2 role (LabInstanceProfile) is used only by the CloudWatch agent; the app makes no AWS API calls and no access keys exist on the servers.
- **Secret handling:** credentials live in src/.env (mode 640, root:apache, outside the web root), written at boot from the launch template; .gitignore excludes it and tests/check_secrets.py scanned 59 tracked files with no credentials found (S18). Known gap: the password is readable by anyone who can view the launch template; Secrets Manager with rotation is the production fix.
- **Network and data access:** private subnets and the security-group chain in Section 2.2; no public IP or SSH on web servers; RDS not publicly accessible; Apache serves only src/public.
- **Data protection:** RDS storage encryption is enabled at rest (StorageEncrypted = true). Data in transit is currently plain HTTP because the Learner Lab does not provide a domain for an ACM certificate, which we accept as a documented risk (Section 7.3).

Named threat validated: Insecure Direct Object Reference (OWASP A01, Broken Access Control), a form of the "data breach" and "insecure interfaces and APIs" risks in the CSA top threats. A logged-in student edits item_id in the URL or the hidden lost_item_id in the claim form to read or claim another student's item. Test 2 tried seven such attacks and none succeeded (Section 6). Other application threats (SQL injection, stored XSS, privilege escalation) are also tested (S13–S15, S09–S10).

5. Analytics or AI/ML feature: data, method, evaluation and limitations

5.1 Dataset and provenance

The matching feature uses a synthetic dataset generated by data/generate_dataset.py with a fixed random seed (42) for full reproducibility. No real student reports, personal data, or photos were used, in compliance with the module's data policy.

Dataset

- **105 lost item reports** (data/lost_items.csv)
- **105 found item reports** (data/found_items.csv)
- **A ground-truth mapping** (data/ground_truth.csv) linking each lost report to its true corresponding found report, where one exists

The dataset spans 8 item categories commonly reported on campus — Bottle, Keys, Card, Bag, Umbrella, Earphones, Jacket, and Watch — with varied phrasings, locations, and dates to approximate realistic reporting behaviour.

We evaluated 105 lost reports against 105 found reports: large enough to contain many same-category items and real ambiguity, small enough to trace every wrong prediction by hand (Section 5.4). Of the 105 lost items, **85 have a genuine corresponding found item** deliberately planted in the found-items set, while **20 have no true counterpart**. We included 20 lost items with no matching found item so we could also test what happens when the correct answer is “no match”. This is important because in a real lost and found system, not every lost item will have been found yet.

Full field definitions and provenance notes are documented in data/DATA_DICTIONARY.md.

5.2 Method

We chose TF-IDF because it is lightweight and can run directly on our existing EC2 instances without needing a GPU or separate AI service. This keeps the setup simpler and avoids extra cloud costs. It also gives a similarity score that is easy for users to understand, since they can see how closely a found item matches their lost item instead of only being shown a yes/no result. For a campus lost and found system with tens or a few hundred active reports, this approach is suitable for our expected workload.

Pipeline: The process has four main steps. First, lost and found reports are stored in RDS. Next, the category, description and location are combined into one text string and common English stop words are removed. TF-IDF is then used to convert the text into vectors, and cosine similarity is used to rank the found items. Finally, the system shows up to five results with a score of at least 0.20. The same get_matches() function is used by both the website and the evaluation script, so the test results reflect what users actually see.

5.3 Evaluation

The engine was evaluated against the ground-truth mapping using the metrics in our success criterion (Section 1.2):

- **Top-k accuracy (k = 1, 3, 5):** checks whether the correct found item appears within the first 1, 3 or 5 suggestions. We mainly use Top-5 because users can see up to five matches.
- **No-match suggestion rate:** checks how often the system still shows a suggestion for the 20 lost items that do not actually have a matching found item.

Metric (python3 analytics/match.py, 2026-10-08)	**Production: category + description + location**	**Baseline before fix: description only**
Top-1 accuracy	34.1% (29/85)	8.2% (7/85)
Top-3 accuracy	65.9% (56/85)	25.9% (22/85)
Top-5 accuracy (target ≥ 50%)	**89.4% (76/85): PASS**	35.3% (30/85): FAIL
No-match items given a suggestion	100% (20/20)	100% (20/20)
Average suggestions shown	4.97 (minimum score 0.20)	4.44

We also tested different minimum similarity scores to see if increasing the threshold would reduce false matches. However, there was no single threshold that solved the problem. With thresholds between 0.20 and 0.30, the Top-5 accuracy stayed at 89.4%, but all 20 items with no real match still received suggestions. Raising the threshold to 0.50 reduced this to 14 out of 20, but Top-5 accuracy also dropped to 64.7%. At 0.60 and above, the system no longer met our target. Because of this, we kept the threshold at 0.20.

Our first evaluation also had a problem. On 24 September, we reported a Precision@1 result of 32.9% using a 0.30 threshold, but this did not actually measure the Top-5 target used in the project. We later found that the live application was also matching descriptions differently from the evaluation script. After fixing matching.php so both used category, description and

location, and fixing a bug where identical descriptions could be linked to the wrong found item, we ran the test again. The final Top-5 accuracy was 89.4%, which passed our target.

Full test records, including the threshold sweep and per-item results, are documented in evidence/test-data-ai.md.

5.4 Limitations and responsible-use considerations

One reason the Top-1 accuracy is not very high is that many items in the dataset have similar descriptions. For example, two students may both describe an umbrella as a “black folding umbrella”, even though they are talking about different items. Because TF-IDF mainly looks at word overlap, these similar descriptions can receive almost the same score. In one case, lost item L21 was matched first to an unrelated black compact umbrella with a score of 0.78, while its actual match, F21, came third with a score of 0.59. This shows one of the main weaknesses of using text similarity by itself.

Other limitations: The synthetic dataset also has some limitations. The wording for lost and found items is generated separately, so some true pairs can even disagree on details such as colour. At the same time, true pairs share the exact same location and date, which may make location more useful in the test data than it would be in real life. Real users would also write descriptions with more variation, including spelling mistakes, abbreviations and personal details that are not present in the generated dataset.

Responsible use: The matching feature is only meant to help users narrow down possible items. It does not decide who owns an item. Security still checks the claimant's answer against the private detail provided by the finder before approving a claim. This is important because the matching score can still be wrong. The matcher also only uses item information and does not use names, emails, or student IDs.

Improvement plan: We could combine text similarity with date and location checks, encourage users to provide distinctive details, and show “no strong match yet” when confidence is low. We could also compare TF-IDF with a small sentence-embedding model to see whether it handles different wording better.

6. Testing, scalability/resilience and monitoring results

6.1 Test Summary

Test	What we did	Expected results	Actual results
1. Functional	Ran tests/run_tests.py, which scripts 17 steps: register (incl. weak password and duplicate email), log in, report found, browse and filter, report lost and view matches, claim, finder view, admin approve and reject, log out.	All scripted steps pass with no errors.	17/17 passed locally and through the ALB. 33/33 functional and security checks passed overall. Evidence: evidence/test-functional.md
2. Security	Logged in as Student A and attacked Student B's items: changed item_id in the URL, edited the hidden lost_item_id and found_item_id in the claim form, plus admin, SQL injection	Both attempts are blocked. No data from User B is shown or changed.	PASS: 0/7 ID-tampering attempts succeeded and all 18 security

	and XSS attempts (S01–S18).		checks passed. The test also detected a deliberately introduced ownership-check fault. Evidence: evidence/test-security.md
3. Data/AI	Ran python3 analytics/match.py: the app's own get_matches() on 105 synthetic lost and 105 found reports, compared with data/ground_truth.csv.	Correct found item in the Top 5 for ≥ 50% of lost items; every suggestion ≥ 20% with a percentage.	PASS: Top-5 accuracy 89.4% (76/85), Top-1 34.1%. All 20 no-match items still received suggestions, which is a known limitation. Evidence: evidence/test-data-ai.md
4. Scale / resilience	Stopped one of two EC2 instances while a probe hit /health.php every 5 s; then ran tests/load_test.py with 30 threads to test the Auto Scaling policy. The normal 50% CPU target was temporarily lowered to 20% for this test after an earlier 50% run did not trigger scaling.	The site stays up, the ALB stops sending traffic to the stopped instance, the ASG launches a replacement, and a CloudWatch alarm fires.	PASS. Part A: site remained available; 142/144 probes returned HTTP 200. Replacement healthy in 3 min 45 s; alarm fired after 4 min 17 s. Part B: CPU peaked at 30.8%, ASG scaled 2→3, and the third instance was healthy after 8.0 min. Evidence: evidence/test-resilience.md

What the results tell us. The deployed workflow and access controls behaved as designed: 17/17 functional steps and all security checks passed. The matcher met its Top-5 target but cannot reliably identify when there is no match, so claims still require human approval. During the resilience test, the site stayed available after one instance was stopped and the CloudWatch alarm met its 5-minute target. Auto Scaling also worked when the CPU threshold was temporarily lowered to 20%, although the high response times showed that CPU utilisation is not the best scaling signal for this application.

6.2 Scalability and resilience mechanism

We use horizontal scaling (scale out/in) rather than vertical scaling because it removes the single point of failure at the web tier. The ALB spreads traffic over lostfound-asg (min 1, desired 2, max 3) across two AZs. The target group checks /health.php every 15 s; the page returns 200 only if PHP can reach the database, so a server that would only show errors is taken out of rotation after two failures (about 30 s). With ELB health checks on, the ASG replaces unhealthy instances automatically (300 s grace period), and a target-tracking policy adds instances when average CPU exceeds 50%. RDS automated backups (1-day retention, point-in-time restore) cover database recovery; the database itself is Single-AZ, so it has no automatic failover.

Test 4 confirmed the mechanism works. For the scaling test, the CPU target was temporarily lowered from 50% to 20% because the normal 50% target did not trigger scaling. With 30 threads, CPU peaked at 30.8% and the ASG scaled from 2 to 3 instances. The third instance became healthy 8.0 minutes after the test started. However, median response time reached 28.7 seconds and 93 of 518 requests timed out, showing that CPU utilisation alone is not a good scaling signal for this workload.

6.3 Monitoring and operational test

CloudWatch collects ALB metrics (UnHealthyHostCount, HTTPCode_Target_5XX_Count, TargetResponseTime), ASG and RDS CPU and RDS free storage. The CloudWatch agent ships Apache access/error logs and the PHP error log from every instance to /lostfound/httpd (7-day retention). The alarm lostfound-unhealthy-alarm fires when any target fails its health check for one minute; it was triggered on purpose in Test 4. A CloudWatch Logs Insights query counts requests per page and status code (evidence/monitoring.md).

During Part A, lostfound-unhealthy-alarm changed from OK to ALARM at 17:34:37 and returned to OK at 17:35:37. The alarm fired 4 min 17 s after the instance was stopped, meeting our 5-minute target. The CloudWatch Logs Insights query scanned 105,452 records. /health.php returned 1,970 successful responses, while 1,452 /matches.php requests returned HTTP 200 and 85 returned HTTP 504 under heavy load. Most requests from the earlier 60-thread run were redirected to the login page because the test login failed. The 504 responses show that the matching process becomes a bottleneck under heavy load.

Limitation: in the Learner Lab the alarm has no SNS notification target; in production it would notify the team by email or chat.

7. Cost, sustainability and operational considerations

7.1 Estimated running cost

The table below shows the estimated monthly cost if the system ran continuously with two instances in the US East (N. Virginia) region (us-east-1), using on-demand pricing.
The estimated cost is approximately USD $95 per month, depending on Application Load Balancer usage, CloudWatch log volume and the number of public IPv4 addresses in use.

Item	Assumption	Estimated monthly cost (USD)
EC2 instances	2 × t3.micro running 24/7	~$15
EBS storage	2 × 8 GB gp3 volumes	~$1.00

Application Load Balancer	1 ALB, very low traffic	~$17-20
RDS MySQL	1 × db.t3.micro, single-AZ, 20 GB storage	~$15
NAT Gateway	1 NAT gateway (US$0.045/h) + small data processing	~$33-35
Public IPv4 addresses	3 public IPv4 addresses (ALB across 2 AZs + NAT Gateway Elastic IP)	~$11
CloudWatch	A few alarms, 7-day log retention	~$1
Total		**~$95 per month**

Cost reasoning: The biggest costs are fixed hourly charges rather than compute: the NAT gateway and the Application Load Balancer. The NAT gateway is the cost of keeping the web servers in private subnets with no public IPs; putting them in public subnets would reduce cost but expose them directly, so we kept the NAT gateway for security. Text-only matching avoids S3 and a paid AI service, while TF-IDF runs on the existing EC2 instances.

We also tried to keep costs down while building the project. When traffic is low, the Auto Scaling group goes down to just 1 instance. We used a single-AZ database since Multi-AZ would cost about double. Logs only stay for 7 days, and we added an AWS Budgets alert so we'd know if we were spending too much. On days when no one was working on it, we stopped the database and temporarily reduced the ASG minimum and desired capacity to 0.

7.2 Sustainability

- **Right-sizing:** We use burstable t3.micro instances because normal traffic is low.
- **Scaling:** The ASG can scale down to one instance when demand is low and could be scheduled to stay at minimum capacity overnight or during term breaks.
- **Lightweight matching:** TF-IDF runs without a GPU or separate AI service, reducing compute and storage needs.
- **Idle shutdown:** Resources were stopped when not being tested and will be deleted after assessment.
- **Future improvement:** Graviton (t4g) instances could reduce both cost and energy use.

7.3 Running this in production

If SIT actually wanted to use this system, there are a few things we'd have to change first:

- **HTTPS:** Add an ACM certificate to the ALB so passwords and personal data are encrypted in transit.
- **Sessions:** Move PHP sessions to RDS or ElastiCache so they survive instance replacement.
- **Matching:** Run matching as a background task when a report is submitted and store the scores.
- **Database:** Enable Multi-AZ and extend backup retention from 1 to at least 7 days.

- **Credentials:** Move database credentials to Secrets Manager with rotation.
- **Identity and claims:** Use SIT SSO, verify student cards at handover, add approval audit logs and a final `returned` status.
- **Privacy:** Define a retention period, such as deleting closed reports after 90 days.
- **Security and operations:** Add CSRF protection, login rate limiting, SNS alarm notifications, CloudTrail and a custom AMI for faster replacement instances.

8. Team contribution, ethical considerations and reflection

8.1 Team contributions
Summary of contributions (full detail in TEAM_CONTRIBUTIONS.md)

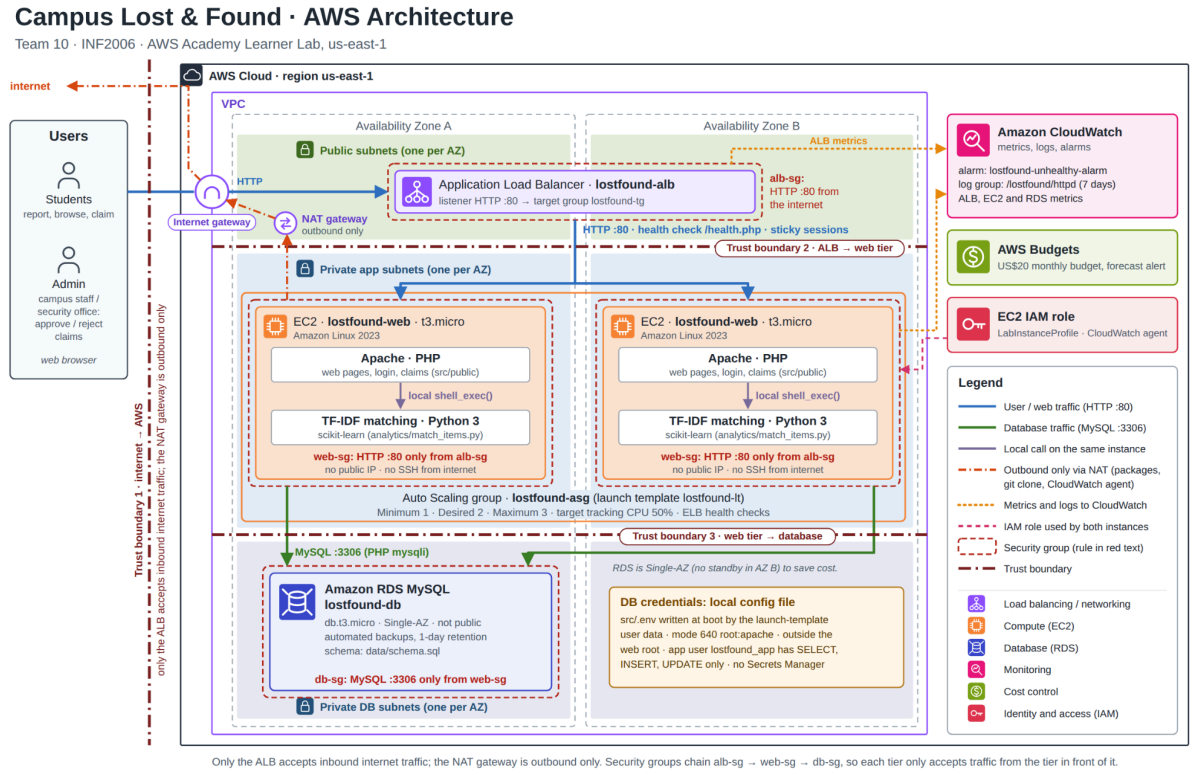
8.2 Ethical considerations
We used synthetic data only, so no real student information was included in testing. Matching only suggests possible items; Security still verifies ownership, and private information is restricted by role. AI-assisted work is recorded in AI_USE_DECLARATION.md

8.3 Reflection
We learned that testing and documentation should happen throughout development. As the project changed, we had to keep the code, AWS setup, tests and report consistent. If we did this again, we would deploy earlier and collect cloud evidence as we worked

References
INF2006 lecture notes 1–5 (Cloud Fundamentals; Service Models; Deployment; Data Storage and Analytics; Security), SIT, 2026. OWASP Top 10 (2021). Cloud Security Alliance, Top Threats to Cloud Computing. AWS documentation: EC2 Auto Scaling, Elastic Load Balancing, Amazon RDS, Amazon CloudWatch. scikit-learn documentation: TfidfVectorizer, cosine_similarity.

Appendix

Figure 1. Architecture of Campus Lost & Found (evidence/architecture.png): users, ALB, Auto Scaling web tier, RDS, the three trust boundaries, security-group rules, data flows and observability.



Appendix B: [Prototype Demonstration Video]

A walkthrough of the deployed Campus Lost & Found prototype, showing the student and admin views, the description-matching feature and the claim workflow.